# CSC 421 - Reinforcement Learning

## Instructor: Shengyao Lu

## Readings

CHAPTER 17 MAKING COMPLEX DECISIONS

- Basic: Sections 17.1 and Summary
- Expected: Sections 17.3
- Advanced: All the chapter including bibliographical and historical notes

CHAPTER 22 REINFORCEMENT LEARNING

- Basic: Sections 22.1 and Summary
- Expected: Sections 22.3
- Advanced: All the chapter including bibliographical and historical notes

## 1. Why Reinforcement Learning (RL)?

AI learns to Walk video

Machine Learning learns a function to solve the task.

- Previously, we learned to solve tasks from a fixed dataset (e.g., learning a classifier or a regression function)
- Now, the agent must **interact with an environment** and **decide a sequence of actions over time**
- Instead of predicting a single output, we need to learn a **strategy (policy)** of how to determine our way to the output

- **Sequential decision-making**
  - actions are taken over time, and **current actions affect future outcomes**
- **No ground-truth labels of actions**
  - it can be impossible to pre-label the "correct" action for every possible scenario

    - or "how good" each action is
    - # of candidate outputs grow exponentially as the plan gets long
  - the agent must learn from **trial and error** (try and see how the strategy works in real-world)
- **Delayed and sparse feedback**
  - rewards may come much later (not after every action)
- **Interaction with the environment**
  - data is not fixed; environment is unpredictable, it depends on the agent's behavior

| Feature | Supervised Learning | Unsupervised Learning | Reinforcement Learning |
|---|---|---|---|
| **Data Labeling** | Requires labeled data | No labels required | Uses rewards and penalties |
| **Objective** | Predict outputs | Identify patterns | Optimize decision-making |
| **Common Algorithms** | Regression, SVMs, Neural Networks | Clustering, PCA, Autoencoders | Q-Learning, DQN, Policy Gradient |
| **Applications** | Healthcare, Finance, Marketing | Cybersecurity, Retail, NLP | Robotics, Gaming, Finance |

*source: Supervised, Unsupervised, and Reinforcement Learning: A Comprehensive Guide*

Reinforcement learning continues to be one of the most active areas of machine learning research. It frees us from manual construction of behaviors and from labeling the vast data sets required for supervised learning, or having to hand-code control strategies.

In other words, in some tasks, your goal is to determine what action to take at each step, but there is no way to identify the "ground truth" of "good actions." Therefore, you cannot label the data for each state. All you have is "experience": after each "try", you only know whether you succeeded or failed. If you succeed, then all the actions you just took may seem to deserve reward; if you fail, then something must have gone wrong, but you cannot tell exactly which action was responsible. As a result, you need to gain experience over and over again. Over time, statistically, the "wrong" actions will receive lower rewards, while the "good" ones will gradually stand out.

- **Core Question of RL:** How to act sequentially under uncertainty to maximize long-term reward?

# 2. Problem Formulation: Markov Decision Process (MDP)

- **MDP**: a sequential decision problem for a fully observable, stochastic environment with a Markovian transition model and additive rewards. Methods for solving MDPs usually involve dynamic programming: simplifying a problem by recursively breaking it into smaller pieces and remembering the optimal solutions to the pieces. It consists of:
  - a set of states (with an initial state $s_0$);
  - a set $\mathrm{ACTIONS}(s)$ of actions in each state;
  - a transition model $P(s'|s,a)$;
  - a reward function $R(s,a,s')$, immediate feedback signal
- **Markov Property:** The future state depends only on the current state and action, not on past history
  - i.e., we can transition model $P(s'|s,a)$ to determine next state.
- **Return (long-term accumulated reward)**:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

  - Discount factor $\gamma \in [0,1]$: balances immediate vs. future rewards
- **Terminal State:** a state after which the interaction ends, so there are no further actions or future rewards.
  - Maze navigation: the agent reaches the exit.
  - Chess: the game ends in a win, loss, or draw.
  - Robot delivery: the robot successfully delivers the item to the target location.
  - Video games: the character dies or completes the level.
- **Episodic vs Continuing:**
  - Episodic task: they end after reaching a terminal state.
    - Playing a game of chess
    - Navigating through a maze to reach a goal
    - Completing one delivery task
    - Finishing one dialogue/session in a task-oriented chatbot
  - Continuing task: they have no natural terminal state and may run forever, hence $\gamma$ is important.
    - Elevator control in a building

    - Traffic light control at an intersection
    - Data center resource management
    - Autonomous driving over long-term operation
    - Stock trading over time
- **Policy** $\pi(\cdot)$ (what to do):
  - A policy defines what action to take in each state, i.e., $a^{(t)} = \pi(s^{(t)})$
  - Goal:

$$\pi^* = \arg\max_{\pi} \ \mathbb{E}[G_t \mid \pi]$$

  - Proper policy: A policy that is guaranteed to reach a terminal state.
- **Value function** $V(s)$: describes state value (how good a state is)

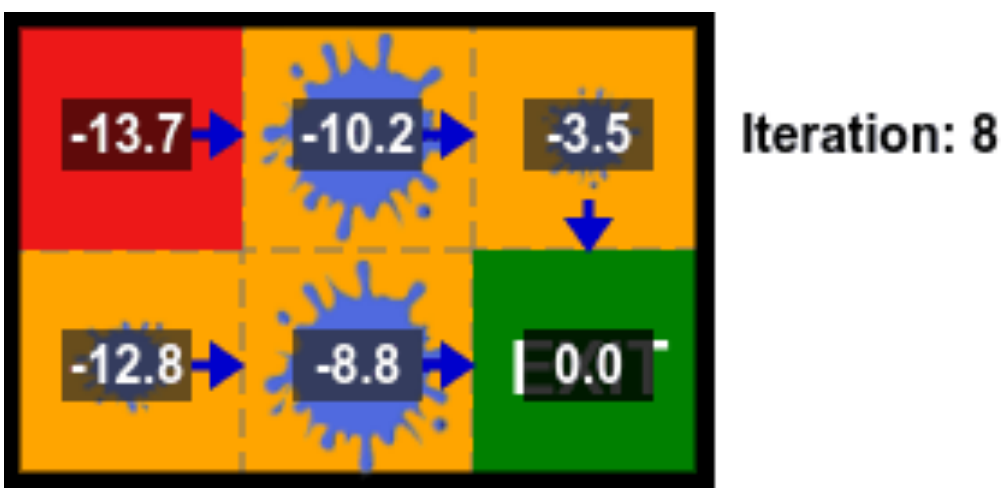
$$V^{\pi}(s) = \mathbb{E}[G_t \mid s_t = s, \pi]$$

- **Q-function (action-value function)** $Q(s, a)$: describes the value of an action
  - It is the expected Return of taking a given action $a$ in a given state $s$.

$$Q^{\pi}(s, a) = \mathbb{E}[G_t \mid s_t = s, a_t = a, \pi]$$

$$V(s) = \max_{a} Q(s, a)$$

$$\pi^*(s) = \arg\max_{a} Q(s, a)$$




## Two different ways to classify RL problems

- Model-based vs. model-free: do we know / learn the environment model?
  - **Model-based:** the agent knows or learns the environment model, such as **transition probabilities** $P(s'|s, a)$ **and state utility functions** $V(s)$,

and can use that model for prediction or planning.
  - **Model-free:** the agent does not explicitly model the environment, and instead **learns action-state values $Q(s, a)$ or policies $\pi(s)$ directly from experience**.
- Passive vs. active RL: is the policy fixed, or does the agent need to learn how to act?
  - **Passive RL**: focuses on **policy evaluation**. The policy is fixed. The agent does not need to decide what action to take. It only needs to learn how good that policy is.
  - **Active RL**: focuses on **policy learning**. The agent must choose actions for itself and learn a good policy from experience.

| | Model-based | Model-free |
|---|---|---|
| **Passive RL** | Policy evaluation with a known model | Direct utility estimation, TD learning under a fixed policy |
| **Active RL** | Value iteration, policy iteration, adaptive dynamic programming | Q-learning, SARSA, REINFORCE, Deep Q-Network(DQN) |

# 3. Exploration vs. Exploitation in RL

The multi-armed bandit (MAB) problem is a well-known challenge in reinforcement learning, decision-making, and probability theory. It involves an **agent repeatedly choosing between multiple actions**, each offering a reward from some fixed probability distribution. The goal is to maximize the return over a fixed number of time steps.

**Multi-armed bandit is simpler than MDP**

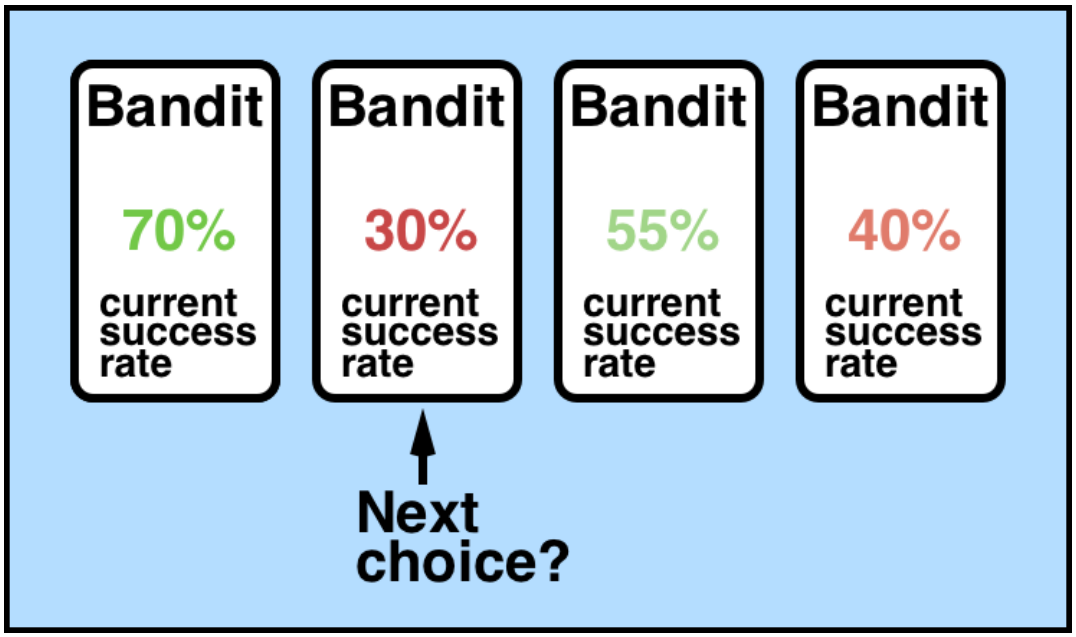
- Each slot machine = one arm, agent chooses one action from several arms
- Each action gives a reward, Reward distribution unknown

- No state transition, Each round is independent except for what the agent has learned
- Goal = maximize total reward over a fixed number of repeated trials

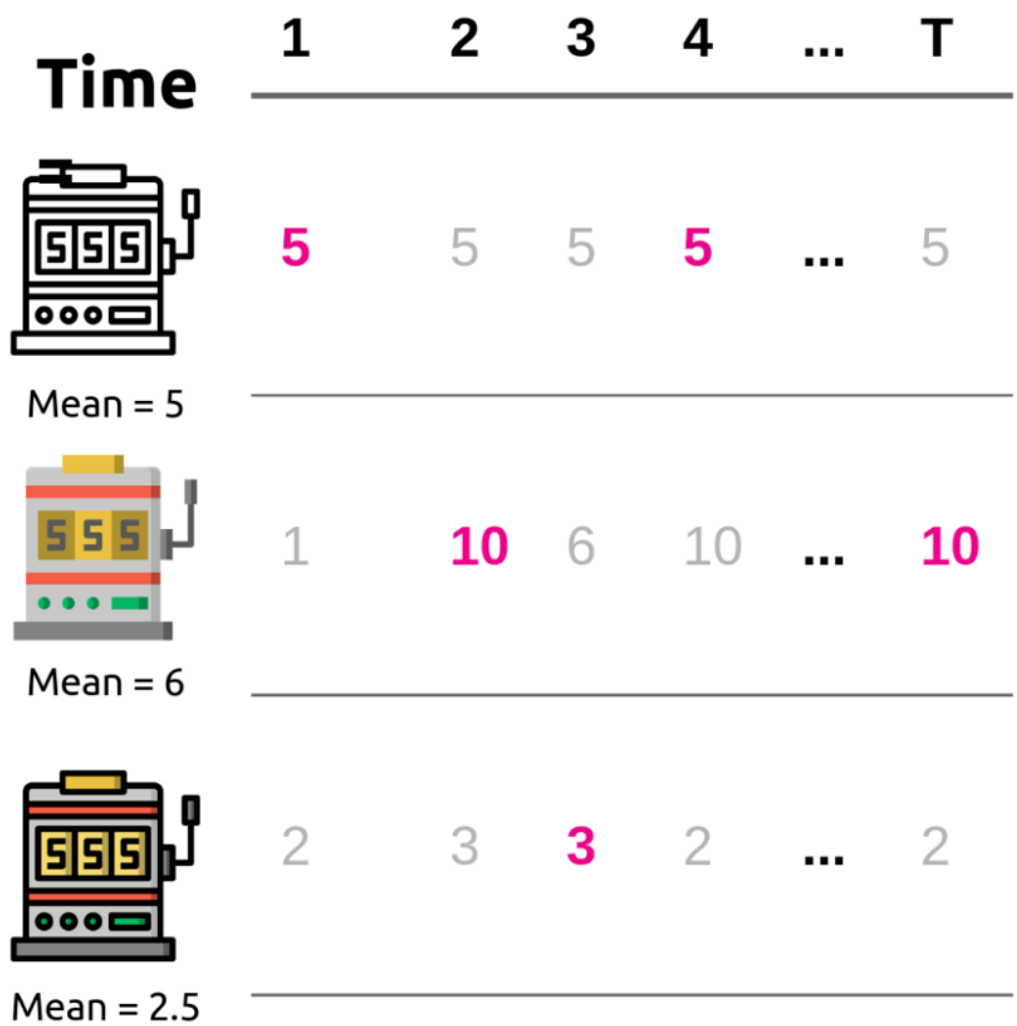
The MAB problem captures the challenge of balancing exploration and exploitation:

- **Exploration:** trying different arms to gather information about their payouts;
- **Exploitation:** pulling the arm that currently seems best to maximize immediate rewards.

A naive approach — playing a single arm repeatedly — might lead to suboptimal returns if a better arm exists but remains unexplored. Conversely, excessive exploration can waste resources on low-reward options.




**Why pure greedy fails?**




source: Introduction to the one-armed bandit model and its use in marketing, numberly

**Approximately optimal bandit policies**

- MCTS (See lecture notes of 03_adversarial_search)

$$a_t = \arg\max_a \left[ \underbrace{Q_t(a)}_{\text{exploitation}} + \underbrace{c\sqrt{\frac{\ln t}{N_t(a)}}}_{\text{exploration}} \right]$$

$$\text{where } Q_t(a) \text{ is the current estimated value of arm } a,$$

$$N_t(a) \text{ is the number of times arm } a \text{ has been selected up to time } t,$$

$$t \text{ is the current time step, and } c > 0 \text{ controls the exploration strength.}$$

# 4. Q-learning

## 4.1 Bellman equations

- By the definition of return, we have:

$$G_t = R_t + \gamma G_{t+1},$$

  i.e., current return = immediate reward + discounted future return
- Given a deterministic policy $\pi(a|s) = 1$ and a deterministic transition function $P(s'|s,a) = 1$, at a state $s$, for certain $a$ and $s'$, the state value:

$$V^\pi(s) = R(s,a,s') + \gamma V^\pi(s')$$

- If neither $\pi(a|s)$ nor $P(s'|s,a)$ is deterministic, the Bellman expectation equation is:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s,a)\big(R(s,a,s') + \gamma V^\pi(s')\big)$$

- Then to obtain the optimal value function, we choose the action that maximizes the expected return.

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s,a)\big(R(s,a,s') + \gamma V^*(s')\big)$$

  That's the Bellman optimality equation.
- **Intuition:**
  - Recursively defined value: Goodness of current state depends on the goodness of next state.
  - Decomposition of optimality: Ensure that when a $\pi$ is optimal at current state, it is also optimal at all future states.

## 4.2 Temporal difference (TD)

- Instead of waiting until the end of an episode, we update an estimate using:

$$new\ estimate \leftarrow old\ estimate + \alpha(target - old\ estimate)$$

  i.e., correct the current estimate using a one-step sample-based target
- For state values, given $(s,a,r,s')$, a one-step TD update is:

$$V(s) \leftarrow \underbrace{V(s)}_{\text{current estimate}} + \alpha(\underbrace{\underbrace{r + \gamma V(s')}_{TD\ target} - \underbrace{V(s)}_{\text{current estimate}}}_{TD\ error})$$

where $r =$ immediate reward, $s' =$ next state.

## 4.3 Q-learning

- Motivation
  - Bellman optimality equation tells us what the optimal value should satisfy
  - but in **model-free RL**, we do not know $P(s'|s,a)$
  - so we cannot compute the expectation directly
  - instead, we use ***one sampled transition*** ($(s,a,r,s')$) to approximate it
- Q-learning update

$$Q(s,a) \leftarrow \underbrace{Q(s,a)}_{\text{current estimate}} + \alpha(\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{target}} - Q(s,a))$$

- Recall:
  - Bellman expectation equation:

$$Q^{\pi}(s,a) = \sum_{s'} P(s' \mid s,a)\big(R(s,a,s') + \gamma \sum_{a'} \pi(a' \mid s') Q^{\pi}(s',a')\big)$$

  - Bellman optimality equation:

$$Q^{*}(s,a) = \sum_{s'} P(s' \mid s,a)\big(R(s,a,s') + \gamma \max_{a'} Q^{*}(s',a')\big)$$

- When we use ***one sampled transition*** $(s,a,r,s')$, Bellman optimality equation becomes

$$Q^{*}(s,a) = r + \gamma \max_{a'} Q^{*}(s',a')$$

- After learning $Q(s,a)$, the agent can choose actions by

$$\pi(s) = \arg\max_{a} Q(s,a)$$